

|                                                                                                                |                                                                                           |
|----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
|  <b>UNIVERSITI<br/>MALAYA</b> | <b>KIE4031</b><br><br><b>Machine Learning</b><br><br><b>FSA 1</b><br><br><b>2025/2026</b> |
|----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|

## Stock Price Prediction Using GRU Architecture (IBM)

| <i>Name</i>                                | <i>Matric Number</i> |
|--------------------------------------------|----------------------|
| Muhammad Rashidul Bukhari Bin Asan Mohamed | 22004502/1           |

Code Link (GitHub): [https://github.com/Bukhari7703/ML\\_FSA1\\_Bukhari.git](https://github.com/Bukhari7703/ML_FSA1_Bukhari.git)

**PREPARED FOR: ASSOCIATE PROF. IR. DR. ANIS SALWA BINTI MOHD  
KHAIRUDDIN**

## Contents

|                                                                                |    |
|--------------------------------------------------------------------------------|----|
| 1 Data Collection And Preprocessing .....                                      | 3  |
| 1.1 Data Sourcing and Univariate Feature Selection .....                       | 3  |
| 1.2 Data Normalization and Scaling .....                                       | 3  |
| 1.3 Temporal Sequencing .....                                                  | 3  |
| 1.4 Tensor Reshaping and Chronological Splitting .....                         | 4  |
| 2 Investigation of the Proposed ML Technique .....                             | 5  |
| 2.1 Gated Recurrent Unit (GRU) .....                                           | 5  |
| 2.2 Architectural & Optimization Characteristics .....                         | 5  |
| 3 Application, Evaluation, and Visualization .....                             | 7  |
| 4 Critical Analysis .....                                                      | 9  |
| 4.1 Strengths of the Base Model (GRU) .....                                    | 9  |
| 4.2 Identified Limitations.....                                                | 9  |
| 4.3 Comparative Analysis with Alternative Architectures (5-year dataset) ..... | 10 |
| 4.4 Comparative Analysis with Alternative Architectures (30-year dataset)..... | 12 |

# 1 Data Collection And Preprocessing

## 1.1 Data Sourcing and Univariate Feature Selection

The core of building the predictive model is built based on reliable past stock data. The stock historical data set of the International Business Machines Corporation (IBM) are used. This data was automatically collected using the `'yfinance'` API from the public finance database on Yahoo Finance. To thoroughly evaluate the neural network architectures, two distinct datasets were extracted which are a 5-year historical dataset to represent a modern, constrained timeline, and an expansive 30-year dataset to test the models' scalability against long-term market noise.

To maintain scientific rigour in our baselining process and prevent overparameterization due to the huge size of this dataset. A univariate forecasting methodology was employed. While multivariate datasets including “Open”, “High”, “Low”, “Volume” and “Adjusted Close” provide broader market context, combining them introduces risks of multicollinearity and would cause the parameters in our neural network to grow exponentially. All columns were filtered except the daily 'Close' price as the sole predictive feature. Figure below shows code for data sourcing and the feature selection.

```
# Download the full dataset
df = yf.download(SYMBOL, period='5y')

# Clean up columns and isolate 'Close' price
if isinstance(df.columns, pd.MultiIndex):
    df_close = df['Close'][[SYMBOL]].rename(columns={SYMBOL: 'Close'})
else:
    df_close = df[['Close']]
```

## 1.2 Data Normalization and Scaling

Financial time-series data inherently contains large variations in absolute scale for an example IBM's price fluctuating from \$110 to over \$300. These large numerical variations will destabilize the gradient descent optimization process if fed directly into the neural network.

To address this, the 'Close' price was normalized using a `MinMaxScaler` to ensure that all of the data was limited to a range between 0 to 1. Scaling ensures that the RNN's internal weights update smoothly and allows the Adaptive Moment Estimation (adam) optimization algorithm converge efficiently without being biased by the absolute magnitude of the stock price. Figure below shows the code for data normalization and scaling.

```
# Scale the data for the neural network
scaler = MinMaxScaler(feature_range=(0, 1))
scaled_data = scaler.fit_transform(df_close.values)
```

## 1.3 Temporal Sequencing

Traditional machine learning models analyse independent tabular data points, whereas RNNs require sequential data to make any mathematical sense of the chronology. Sliding window approach was used to transform the normalized 1-dimensional array of prices into a form

suitable for supervised learning. A sequence length of 60 timesteps was set, which corresponds roughly to a period of 1 financial quarter.

The model analyses the past 60 days input feature, (X) to forecast the closing price on the 61st day, (y). Then the algorithm slides the window forward by exactly one day, dropping the oldest historical day and appending the newest, to dynamically generate the next continuous sequence. Figure below shows the code for temporal sequencing.

```
# Create 60-day sequences
sequence_length = 60
X, y = [], []

for i in range(sequence_length, len(scaled_data)):
    X.append(scaled_data[i-sequence_length:i, 0])
    y.append(scaled_data[i, 0])

X, y = np.array(X), np.array(y)
```

## 1.4 Tensor Reshaping and Chronological Splitting

Although this sliding window technique successfully engineers the required historical context, the resulting raw arrays are not yet compatible with the deep learning framework. Deep learning frameworks such as TensorFlow strictly require recurrent input data to be formatted as a 3D tensor.

In order to meet this architectural requirement, the output sequences were reshaped into dimension of samples, time steps and features consecutively.

For this model, this resulted in a configuration of (1196, 60, 1), meaning there were 1,196 usable samples, each consisting of 60-time steps, and one feature (the scaled daily 'Close' price). The sequential dataset was then separated into an 80% training set and 20% test set. This is done to avoid the issue of data leakage ensuring the model is trained entirely on past data and evaluated purely on its ability to forecast an unseen, chronological future. Figure below shows the code for tensor reshaping and chronological splitting.

```
# Reshape to 3D Tensor (samples, time steps, features)
X = np.reshape(X, (X.shape[0], X.shape[1], 1))

# Split into 80% Training and 20% Testing to prevent data leakage
split = int(len(X) * 0.8)
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]
```

## 2 Investigation of the Proposed ML Technique

### 2.1 Gated Recurrent Unit (GRU)

Financial time-series forecasting requires models capable of mapping long-term dependencies. Standard RNNs cannot perform such task due to the vanishing gradient problem, where they forget earlier historical data over long sequences. During backpropagation, the gradients used to update the network weights are continuously multiplied. Because these values are typically less than 1, they exponentially shrink toward zero, effectively causing the network to forget earlier historical data over long sequences.

To resolve this mathematical limitation, the **Gated Recurrent Unit (GRU)** architecture was selected. The GRU is computationally efficient because it dynamically controls the flow of information using two key mathematical gates, rather than relying on a heavy, separate memory cell. Unlike the Long Short-Term Memory (LSTM) network, which relies on a heavy three-gate system (Input, Output, and Forget) and a separate cell state, the GRU merges the cell state and hidden state, achieving highly comparable accuracy with significantly higher computational efficiency and fewer trainable parameters.

The flow of temporal information is governed by the following mechanisms:

- **Update Gate ( $z_t$ ):** Controls how much past momentum should be kept and passed to the future.

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

- **The Reset Gate ( $r_t$ ):** Enables how much stale information should be actively forgotten, which is critical during sudden market regime shifts.

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

- **Linear Additive State ( $h_t$ ):** The final hidden state computed using a linear combination of the previous state and the new candidate state,  $\tilde{h}_t$ . This additive operation allows gradients to flow linearly backward through time without vanishing, preserving deep sequence memory

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

### 2.2 Architectural & Optimization Characteristics

The base model was constructed as a sequentially stacked GRU architecture via TensorFlow and Keras framework to extract hierarchical representations of the IBM dataset.

- **Hierarchical Extraction:** The network utilizes a stacked layer of GRU(64) and GRU(32). The first layer analyses the full 60-day sequence, while the second compresses those features to isolate dominant macro-trends from daily market noise.

- **Regularization via Dropout:** Deep networks easily overfit on small datasets. Dropout(0.2) layers were injected to randomly deactivate 20% of the neurons per epoch, preventing the model from memorizing specific training sequences.
- **Dimensionality Reduction:** The derived temporal features are flattened via a Dense(16) fully connected layer that act as a regression head. Then it is passed to Dense(1) layer to produces a continuous scalar output which is the stock price prediction.
- **Dynamic Training Controls:** The model is optimized using the Adam (Adaptive Moment Estimation) algorithm. Adam is uniquely suited for noisy financial data because it computes individual adaptive learning rates for different network parameters using estimates of first and second moments of the gradients. The network was instructed to minimize the Mean Squared Error (MSE) loss function, which exponentially penalizes large outlier errors.
- **Early Stopping:** To dynamically counter overfitting, an EarlyStopping callback was implemented. Financial models easily fall into a "double-descent" trap where validation error spikes while training error drops. By monitoring the validation loss with a patience of 10 epochs, the callback halts the optimization process the moment the model begins memorizing noise, actively restoring the most generalized and mathematically sound weights.

### 3 Application, Evaluation, and Visualization

The predictive models were developed using Python, specifically leveraging the **TensorFlow** and **Keras** deep learning frameworks. These tools were selected for their robust ability to construct, train, and optimize complex recurrent architectures efficiently. To maintain strict scientific reproducibility and ensure that the performance variances between the architectures were structurally driven rather than the result of random stochastic weight initialization, a global random seed of 90 was explicitly set across the training pipeline. Furthermore, before compiling each new model iteration, the computational graph was reset using `tf.keras.backend.clear_session()` to prevent memory leaks or residual state contamination.

The model was compiled using the Adam optimization algorithm which minimize the MSE loss function. During training, the batch size was set to 32 sequences. To rigorously evaluate the model's ability to generalize, the model was trained with 80% of the chronological timeline and tested on the remaining 20%. To prevent the model from overfitting to the training data, an Early Stopping callback was implemented with a patience of 10 epochs.

The standard GRU was deployed as the primary architectural baseline for the 5-year dataset. As illustrated in Figure 1, the model successfully halted training at Epoch 64. The resulting loss curves exhibit a rapid initial reduction in error, followed by a clear, sustained stabilization of the loss function over the subsequent epochs. This trajectory confirms that the network achieved optimal mathematical convergence across the 64 epochs, effectively extracting the underlying market macro-trends without overfitting to the stochastic noise of the training data.

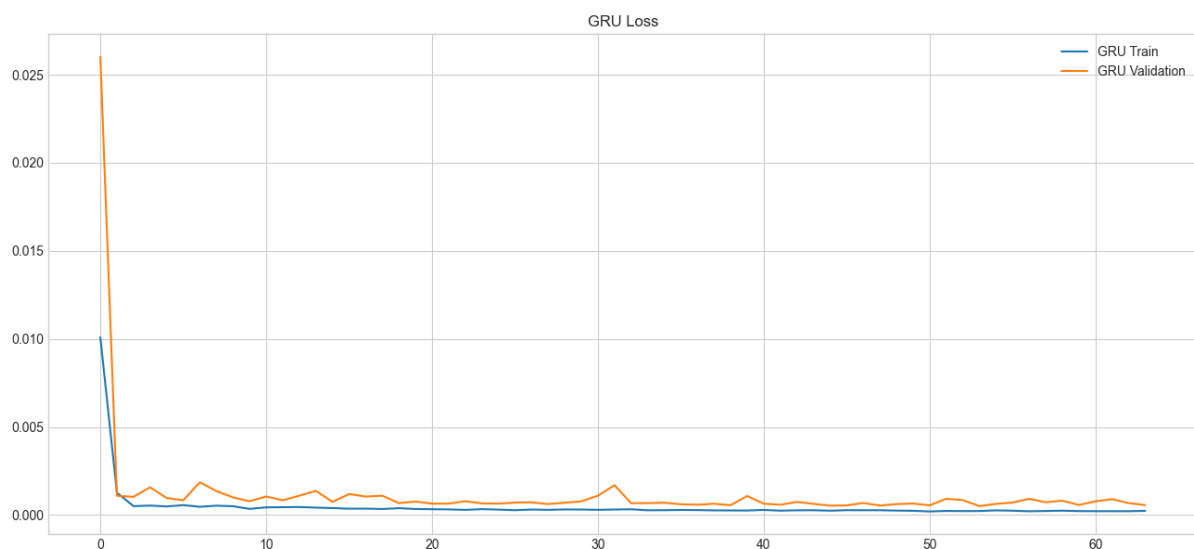


Figure 1: GRU Loss Curve

Furthermore, as shown Table 1 below, the architecture proved computationally efficient, completing the 64-epoch training cycle in 73.36 s, averaging a highly optimized 1.15 s per epoch. The R2 score of 0.8909 proves that the model successfully explains 89% of the variance in IBM's closing price. Additionally, the RMSE indicates that, on average, the model's

predictions deviate by RMSE of \$9.05 from the actual closing price. It is a strong margin of accuracy given the high absolute price fluctuating from \$110 to over \$300.

Table 1: GRU Performance

| Model | Epochs | Training Time (s) | Time/Epoch (s) | MAE  | MSE   | RMSE | R <sup>2</sup> |
|-------|--------|-------------------|----------------|------|-------|------|----------------|
| GRU   | 64     | 73.36             | 1.15           | 6.09 | 81.94 | 9.05 | 0.8909         |

To contextualize the quantitative metrics, the model’s predictions were inverse-transformed and plotted on a graph alongside the historical closing prices for IBM as shown in Figure 2. The predicted values (green line) closely track the actual market trajectory (orange line). The GRU was able to identify the major regime changes, such as the significant decline in mid-2025, the following recovery in 2026, and the local movements without falling into smooth drift.

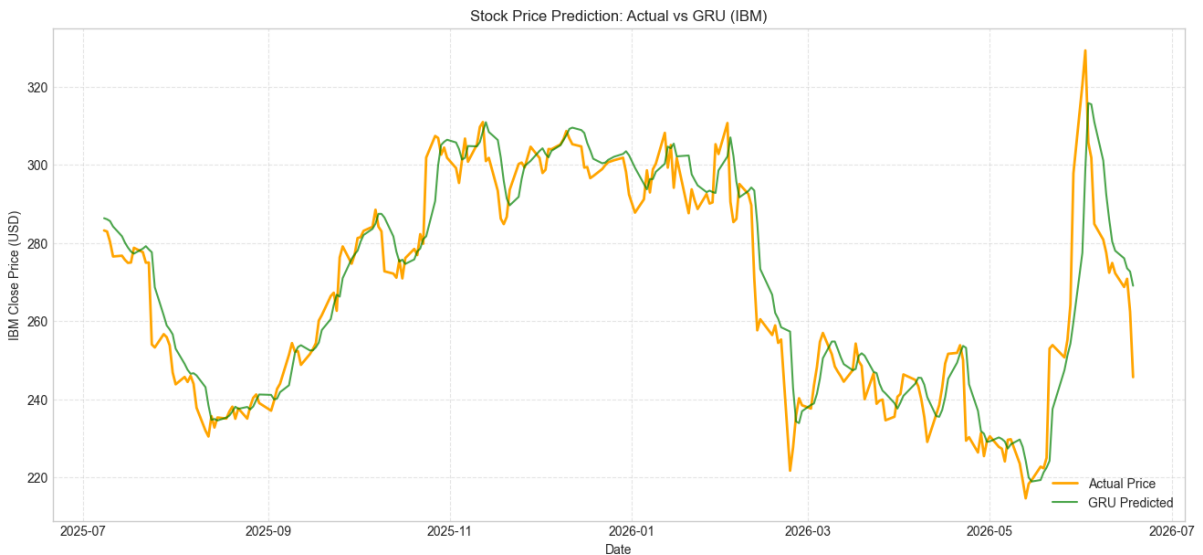


Figure 2: Actual vs Predicted (GRU)



## 4 Critical Analysis

The baseline GRU model for 5-year dataset in this work demonstrated highly robust predictive capabilities for financial time-series forecasting. However, applying deep learning to financial markets requires a critical evaluation of the model's architectural trade-offs, inherent limitations, and performance relative to alternative neural networks.

The strengths, limitations, and a comparative analysis between different models on the 5-year dataset and also another comparative analysis between different models on the 30-year dataset are detailed in this section.

### 4.1 Strengths of the Base Model (GRU)

The GRU demonstrated robust predictive capabilities, confirming its theoretical advantages over other neural network architectures.

#### a) Superiority over Traditional RNNs

Traditional RNN suffer severely from the vanishing gradient problem, rendering them incapable of retaining memory over long sequences. The GRU successfully resolves this by utilizing its Update and Reset gates to dynamically maintain relevant long-term momentum while discarding obsolete market noise.

#### b) Efficiency and Accuracy over Standard LSTM

Although both LSTM and GRU architectures were designed to handle long-term dependency, the GRU model had a clear advantage over the LSTM model in terms of the performance achieved with this particular financial dataset. As visually summarized in Figure 3 and detailed in Table 2, it is evident that GRU model clearly performed much better than the standard LSTM model by obtaining an  $R^2$  value of 0.8909 compared to 0.7455 and significantly reducing the RMSE to 9.05 from 13.82. The fact that the GRU operates with two gates instead of three in the case of LSTM allows it to optimize with few parameters.

#### c) Deployability against Advanced Models

As shown in Table 2, while hybrid architectures such as the CNN-GRU and BiGRU achieved higher  $R^2$  scores, the standard GRU offers critical advantages for real-world deployment. Rooted in the **principle of parsimony**, the standard GRU achieves strong baseline accuracy without the need for complex spatial filters or bidirectional processing. Unlike the BiGRU, which unrealistically reads historical sequences backward and requires significantly more computational training time, the standard GRU predicts solely on forward momentum. This preserves the strict chronological causality strictly required for live trading scenarios.

### 4.2 Identified Limitations

Despite the strong statistical performance of the baseline model, deep learning applied to financial time-series forecasting carries inherent limitations that must be critically evaluated.

#### a) Overfitting Risks on Small Data

Financial datasets have a very low signal-to-noise ratio. A 5-year historical window only generated 1,196 usable training sequence, the neural network has a strong tendency to memorize market noise rather than learning the actual underlying pattern.

b) Data Dependency and Univariate Blindness

The model's predictions are entirely dependent on a single feature which is past closing prices. Because it is a univariate model, it does not factor in trading volume, intraday volatility, macroeconomic indicators, or fundamental company news, which strictly limits its holistic predictive power.

c) Sensitivity to Market Volatility

During sudden, massive price shocks such as the extreme volatility leading into mid-2026 for base model as shown in Figure 2. The model exhibits a noticeable lag. It successfully tracks the trajectory of the volatility but struggles to anticipate the absolute peak of a sudden spike or the bottom of a crash until after the market regime has already shifted.

### 4.3 Comparative Analysis with Alternative Architectures (5-year dataset)

To contextualize the performance of the base GRU, it was benchmarked against three alternative architectures: **LSTM**, hybrid **CNN-GRU** and Bidirectional GRU (**BiGRU**). As shown in Figure 3, these architectures yielded distinctly different balances of accuracy and efficiency.

a) Alternative 1: LSTM

The LSTM served as an underperforming baseline for this study, yielding the lowest  $R^2$  of **0.7455** and the highest RMSE of **13.82**, highlighting the need for more advanced recurrent gating mechanisms when processing this specific dataset. The model follows the actual market value roughly, failing to map the tight local variance accurately as shown in Figure 4.

b) Alternative 2: BiGRU

By running two GRU layers simultaneously, one reading the 60-day sequence chronologically, and the other reading it in reverse, the network maximized its contextual understanding of the market momentum. As shown in Figure 5, BiGRU tracks extreme market volatility and sudden price dips with exceptional tightness, achieving a peak  $R^2$  of **0.9227**. However, this accuracy comes at the cost of severe computational overhead. As illustrated in Figure 3, the model requires **74.06 s** over **100 epochs** to train, making it the most resource-intensive model evaluated.

c) Alternative 3: CNN-GRU

CNN-GRU proved to be the absolute optimal architecture achieving peak performance metrics across all the models. By utilizing a spatial noise filter to mathematically smooth out erratic daily jumps before recurrent processing, CNN-GRU achieved an impressive  $R^2$  of **0.9239** while being the lowest RMSE and MAE of **7.56** and **4.98**. Furthermore, the model

requires one of the shortest time to train which is **36.01 s** and **0.60 s/epoch**. As shown in Figure 6, this filtering allows the model to capture the macro-trend flawlessly without getting trapped in daily micro-fluctuations.

Summary

In 5-year dataset model, **CNN-GRU** and **BiGRU** emerged as the top performers, achieving near-identical, highly accurate predictive scores,  $R^2$  of **0.9239** and **0.9227** respectively. However, **CNN-GRU** demonstrated vastly superior computational efficiency, completing its training in **36.01 s** roughly half the time of the **BiGRU** of **74.06 s**. The standard **GRU** established a solid, lightweight baseline while the complex three-gate architecture of the traditional **LSTM** struggled significantly, yielding the lowest accuracy and the highest error margin.

Table 2: Performance comparison of ML models (5-year dataset)

|      | Model   | Epoch | Training Time (s) | Time/Epoch (s) | MAE  | MSE    | RMSE  | R <sup>2</sup> |
|------|---------|-------|-------------------|----------------|------|--------|-------|----------------|
| Base | GRU     | 64    | 73.36             | 1.15           | 6.09 | 81.94  | 9.05  | 0.8909         |
|      | LSTM    | 31    | 24.53             | 0.79           | 9.57 | 191.11 | 13.82 | 0.7455         |
|      | BiGRU   | 100   | 74.06             | 0.74           | 5.03 | 58.03  | 7.62  | 0.9227         |
|      | CNN-GRU | 60    | 36.01             | 0.60           | 4.98 | 57.17  | 7.56  | 0.9239         |

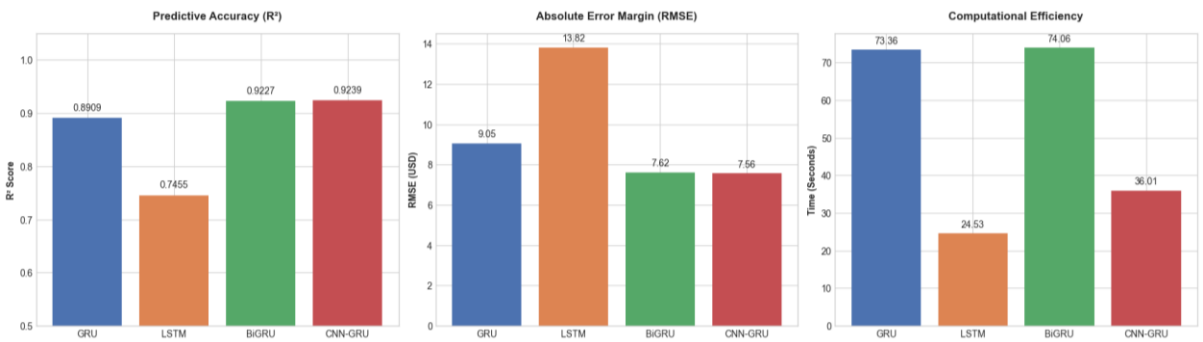


Figure 3: Important metrics comparison of ML model (5-year dataset)

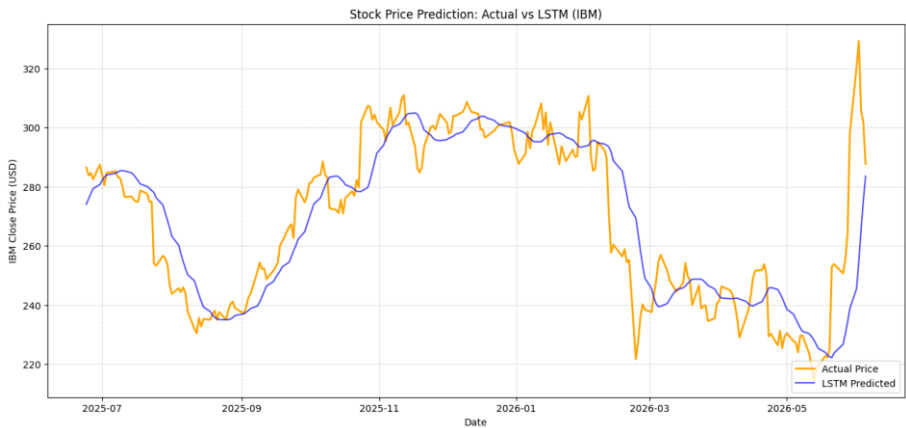


Figure 4: Actual vs Predicted (LSTM)

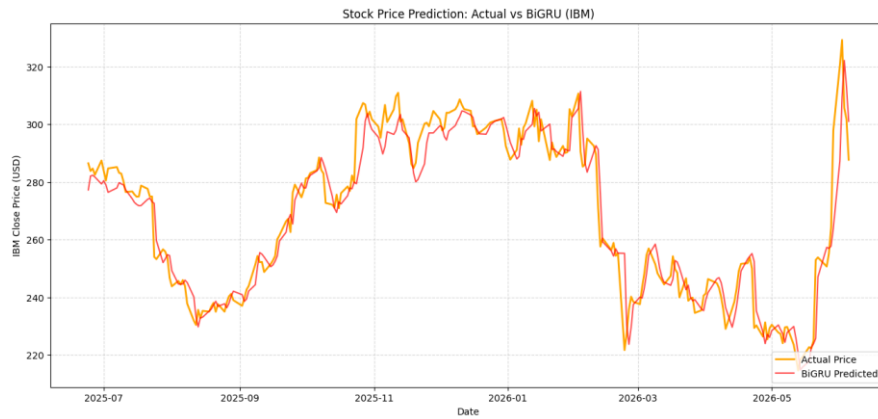


Figure 5: Actual vs Predicted (BiGRU)

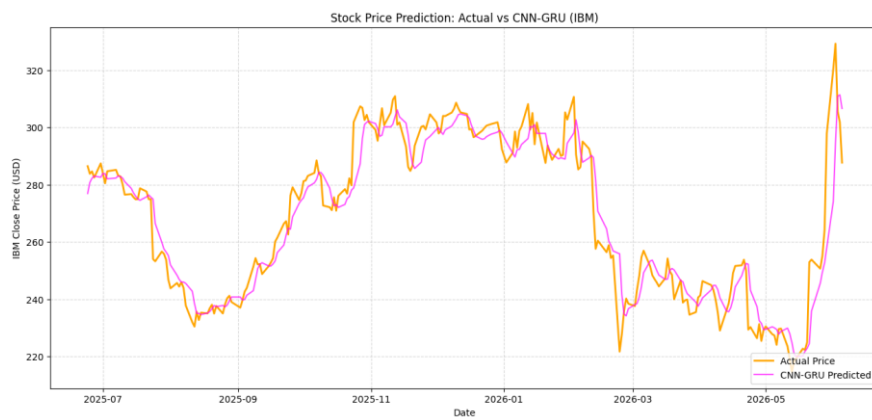


Figure 6: Actual vs Predicted (CNN-GRU)

## 4.4 Comparative Analysis with Alternative Architectures (30-year dataset)

To rigorously test the architectural scalability of the models, it was subjected to a 30-year dataset. The model is subjected to a 30-year dataset in Table 3 to benchmark with current 5-year dataset in Table 2.

Table 3: Performance comparison of ML models (30-year dataset)

| Model   | Epochs | Training Time (s) | Time/Epoch (s) | MAE  | MSE    | RMSE  | R <sup>2</sup> |
|---------|--------|-------------------|----------------|------|--------|-------|----------------|
| GRU     | 19     | 161.16            | 8.48           | 4.48 | 48.06  | 6.93  | 0.9886         |
| LSTM    | 19     | 255.33            | 13.44          | 8.19 | 153.55 | 12.39 | 0.9636         |
| BIGRU   | 28     | 190.64            | 6.81           | 4.53 | 44.25  | 6.65  | 0.9895         |
| CNN-GRU | 40     | 181.20            | 4.53           | 3.91 | 33.08  | 5.75  | 0.9921         |

### a) Alternative 1: LSTM

Across both datasets, the LSTM served as an underperforming baseline. When scaled to the 30-year dataset, its complex three-gate architecture was completely overwhelmed by the continuous multi-decade financial data. It yielded the lowest R<sup>2</sup> of **0.9636** and the highest error

margin of **12.39**. Furthermore, it suffered from severe parameter bloat, requiring an agonizing **255.33 s** to train, averaging a highly inefficient **13.44 s/epoch**. This definitively confirms that the standard LSTM is structurally heavier and less efficient than GRU-based mechanisms for multi-decade time-series forecasting.

b) Alternative 2: BiGRU

On the 5-year dataset, the BiGRU maximized short-term context effectively. When scaled to the 30-year dataset, it successfully maintained high contextual accuracy, achieving an excellent  $R^2$  of **0.9895** and edging out the standard GRU of  $R^2$  of **0.9886**. However, reading three decades of daily sequences in both forward and reverse chronological order incurs a massive mathematical penalty. The model required **190.64 s** to complete its training, making it highly resource-intensive and proving that pure contextual processing scales poorly when handling decades of market history.

c) Alternative 3: CNN-GRU

The CNN-GRU proved to be the absolute optimal architecture when exposed to 30 years of data. A multi-decade financial history contains immense amounts of erratic market noise. The CNN-GRU's 1D Convolutional layer acts as a spatial filter, mathematically compressing decades of daily micro-fluctuations into clean, dominant macro-trends before the recurrent processing even occurs. Consequently, the CNN-GRU achieved the highest  $R^2$  of **0.9921** and the lowest RMSE of **5.75**. Because the recurrent gates are fed cleanly filtered data, the architecture is vastly lighter mathematically, it maintained the fastest computational speed per epoch (**4.53 s/epoch**) despite running for a full 40 epochs.

Summary

The **CNN-GRU** separated itself as the undisputed champion by utilizing a spatial filter to mathematically compress decades of daily micro-fluctuations before recurrent processing, it achieved near-perfect accuracy,  $R^2$  of **0.9921** and the lowest error margin. The standard **GRU** also scaled exceptionally well, leveraging the massive data volume to reach a highly robust  $R^2$  of **0.9886** while converging the fastest. Conversely, the **BiGRU** incurred a massive mathematical penalty in training time by reading decades of sequences backward, and the **LSTM** was structurally overwhelmed by the multi-decade volume, resulting in severe parameter bloat and the poorest predictive accuracy.

Summary: Overall Comparison (5-Year vs. 30-Year Datasets)

Scaling the dataset from 5 to 30 years reveals a critical law of financial forecasting. In the 5-year dataset model, BiGRU and CNN-GRU both achieved top-tier accuracy, though the CNN-GRU was significantly faster. However, when exposed to 30 years of massive dataset, pure contextual processing became a liability. Ultimately, the **CNN-GRU** emerged as the definitive champion across both dataset sized, by mathematically compressing erratic volatility before recurrent processing, it perfectly balanced elite predictive power with superior computational efficiency, outclassing the base class GRU, BiGRU and LSTM.